عشان نفهم الـ Web شغال إزاي بعمق، لازم نتخيلها كأنها عملية "طلب وتنفيذ" بتم في أجزاء من الثانية بين جهازك (Client) وبين جهاز تاني بعيد (Server).

الموضوع بيمشي في خطوات محددة، وكل خطوة ليها دور حيوي:

---

1. الـ Request: البداية من عندك

لما بتكتب `www.google.com` في المتصفح، أنت كدة بتبعت **HTTP Request**. المتصفح هنا هو الـ **Client**، وهو اللي بيبدأ الكلام. الطلب ده مش مجرد اسم الموقع، ده بيبقى معاه تفاصيل تانية زي نوع المتصفح بتاعك واللغة اللي بتفضلها.

2. الـ DNS: المترجم العبقري

الأجهزة على النت مش بتعرف الأسماء (Domain Names)، هي بتعرف أرقام بس اسمها **IP Addresses**.

- المتصفح بياخد الاسم اللي كتبته ويروح يسأل الـ **DNS Server**: "يا عمو، `google.com` ده عنوانه كام؟".
- الـ DNS بيرد برقم زي `172.217.1.1` . ده العنوان الحقيقي للسيرفر اللي عليه الموقع.

---

3. الـ TCP/IP: الاتصال المضمون

قبل ما أي داتا تتبعت، لازم يحصل "اتفاق" بين جهازك والسيرفر إنهم هيفهموا بعض، وده بيتم عن طريق بروتوكول الـ **TCP**.

- بيحصل حاجة اسمها **Three-Way Handshake**: جهازك يبعت سلام، السيرفر يرد بسلام، وجهازك يأكد إنه استلم الرد. كدة "الماسورة" اتفتحت وجاهزة لنقل البيانات.

4. الـ HTTP: لغة الحوار

الـ **HTTP** هو القوانين اللي بتحكم شكل الرسالة اللي مبعوتة. الطلب بتاعك بيبقى فيه:

- **Method:** زي `GET` (عايز أشوف صفحة) أو `POST` (عايز أسجل بيانات).
- **Path:** زي `/login` أو `/products` .
- **Headers:** معلومات إضافية زي الـ `User-Agent` .

---

5. الـ Server Side (شغل الـ Node.js)

هنا بقى السيرفر بيستلم الطلب. لو شغالين بـ Node.js، الـ **Event Loop** بياخد الطلب ده يوديه للـ Code بتاعك:

- الكود بيشوف أنت عايز إيه.

- لو محتاج بيانات، بيروح يكلم الـ **Database**.
- بيجهز الـ **Response** (سواء كان ملف HTML، أو JSON داتا، أو حتى صورة).

6. الـ Response: الرد وصل

السيرفر بيبعت الرد ومعاه **Status Code** (كود الحالة):

- **200 OK:** كل شيء تمام، خد الداتا أهي.
- **404 Not Found:** الصفحة اللي بتدور عليها مش عندي.
- **500 Internal Server Error:** السيرفر حصل فيه مشكلة وهو بيعالج طلبك.

7. الـ Rendering: الشكل النهائي

المتصفح بيستلم الرد (غالباً HTML/CSS/JS). يبدأ يقرأ الـ HTML عشان يبني هيكل الصفحة، والـ CSS عشان يلونها، والـ JS عشان يخليها تفاعلية (زي الزراير والأنيميشن).

الخلاصة للمعمارية (Architecture)

الـ Web عبارة عن **Client-Server Architecture**. الـ Client بيطلب، والـ Server بينفذ ويرد. في Node.js، إحنا بنبني الـ Server اللي بيستقبل الطلبات دي ويتعامل معاها بسرعة عالية جداً بسبب الـ Non-blocking I/O.

---

1. الـ `require("http")` (استدعاء الكبير)

هنا أنت بتقول للـ Node: "يا عمهم، هات لي موديول الـ HTTP اللي عندك". ده موديول أساسي (Built-in) هو المسؤول عن فتح القنوات اللي بنستقبل وبنبعت منها الداتا عبر الإنترنت.

2. الـ `http.createServer()` (بناء المطبخ)

دي أهم مكنة في الـ Node.js.

- الميثود دي بتاخد منك حاجة اسمها **Request Listener**، وهي عبارة عن Function بتشتغل أوتوماتيك أول ما أي "زبون" (Client) يبعت طلب للسيرفر بتاعك.
- الـ Function دي بتديك حاجتين (Arguments) هما أساس اللعبة:
- `req` **(Request):** ده "شنطة" مليانة داتا جاية من الزبون (عنوانه، هو عايز إيه، بياناته).
- `res` **(Response):** ده "الطبق" اللي أنت هتحط فيه الرد بتاعك وتبعته للزبون.

3. الـ `console.log(req)` (كشف المستور)

لما تشغل السيرفر ده وتفتح المتصفح على `localhost:3000` وتدوس Enter، الـ Terminal عندك هيتغرق داتا.

- الـ `req` ده Object ضخم جداً. الـ Node.js بيطلعهولك "على بياض" فيه كل تفاصيل الاتصال.
- **Deep Dive:** في المرحلة دي، أنت بتشوف الـ **IncomingMessage** Object. هتلاقي جواه الـ `method` والـ `url` والـ `headers` (اللي اتكلمنا عليهم في الـ Web works).

4. الـ `server.listen(3000)` (فتح الباب)

السيرفر طول ما هو شغال في الذاكرة مش هيعمل حاجة غير لما "يسمع" (Listen).

- أنت هنا حجزت "بوابة" رقم **3000** على جهازك.
- الـ Node.js هيفضل مشغل الـ **Event Loop** ومش هيقفل البرنامج طول ما هو مستني طلبات تيجي على البوابة دي.

- (أنت فاتح منين) `host` أنت هنا بتطبع "كارت التعارف" اللي جاي من المتصفح. هتلاقي فيه معلومات زي : `req.headers` و `user-agent` (نوع متصفحك).
- السطر ده حيوي جداً. أنت بتبلغ المتصفح: "يا بطل، اللي : `res.setHeader('Content-Type', 'text/html')` فكّكه واعرضه كصفحة ويب، HTML جاي لك ده مش مجرد نص، ده كود".
- الـ (Chunks). على أجزاء HTML دي زي "الترزي" اللي بيفصل القماش حتة حتة. أنت بتبعت الـ : `res.write()` وبيعتها للسلك **Buffer** بيجمع الأجزاء دي في الـ Node.js.
- ومش هيعرض (Loading) دي "الإمضاء" الأخيرة. من غير السطر ده، المتصفح هيفضل مستني ومفتوح : `res.end()` أنت بتقفل "الماسورة" وبتقول للـ Node: "خلاص، كدة المصلحة حاجة لأنه فاكر إن لسه فيه داتا جاية. بمجرد ما بتنادي `end` خلصت".

شرح الكود سطر بسطر (التحليل العميق)

1. `const body = [];`

- **لزمتها:** دي "السبت" اللي هنجمع فيه عربيات القطر (Chunks) اللي بتوصل واحدة ورا التانية. بنبدأ بـ Array فاضي عشان نجمع فيه الحتت دي.

2. `req.on("data", (chunk) => { ... });`

- **القاعدة العامة:** الـ `req` (الطلب) في نود هو عبارة عن **Readable Stream**. والـ `on.()` دي عبارة عن **Event Listener**.
- **المعنى:** أنت بتقول للنود: "يا نود، كل ما تستلم حتة داتا (Chunk) من المتصفح، شغل الـ Function دي".
- **الـ `chunk` :** ده بيبقى عبارة عن **Buffer** (داتا خام بالـ Hexadecimal). بنرمي الحتة دي في السبت بتاعنا `body.push(chunk)` .

3. `req.on("end", () => { ... });`

- **المعنى:** "يا نود، أول ما المتصفح يخلص بعت كل الداتا والقطر كله يوصل، شغل الـ Function دي". دي لحظة التجميع النهائي.

4. `const parsedBody = Buffer.concat(body).toString();`

- **Buffer.concat(body):** بنلزق كل الحتت اللي جمعناها في السبت جنب بعض عشان يبقوا كتلة واحدة.
- **.toString():** عشان نعرف نقراها. هتطلع لك حاجة شبه (String) الداتا كانت "بايتات" (أرقام)، بنحولها لنص مفهوم `message=hello+world` .

5. `const message = parsedBody.split("=")[1];`

- **المعنى:** الداتا اللي جاية من الـ Form بتبقى بصيغة `key=value` . إحنا بنقسم النص عند علامة `=` وبناخد الجزء التاني `[1]` اللي هو "القيمة" اللي اليوزر كتبها فعلاً.

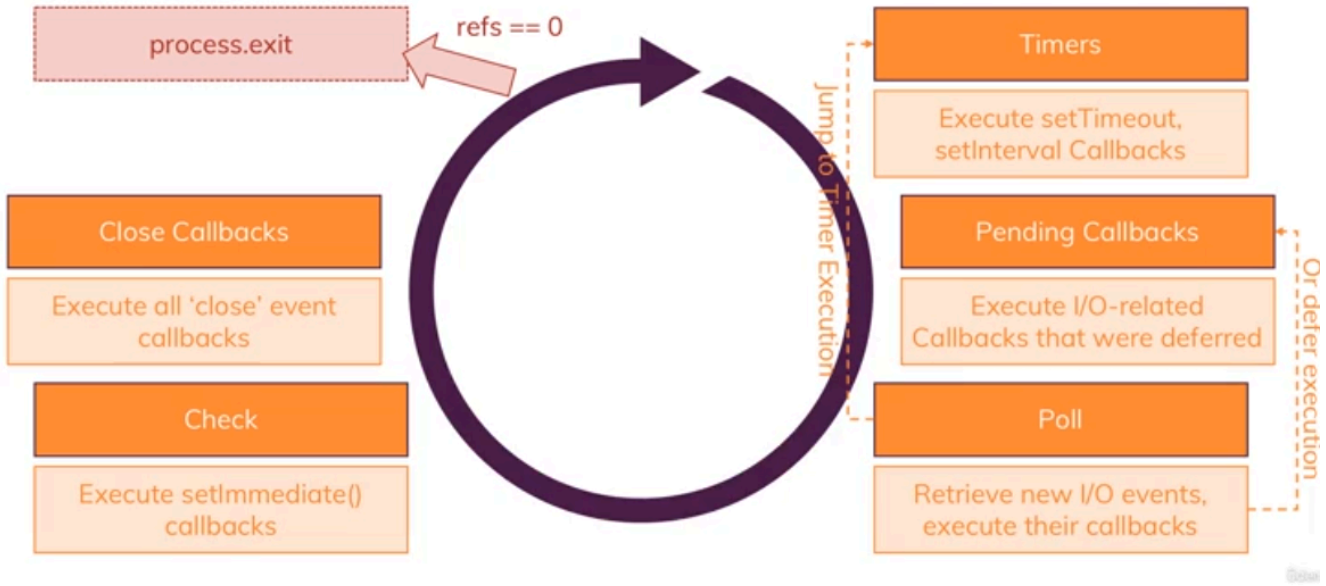
6. `fs.writeFileSync("message.txt", message);`

- **المعنى:** سجل يا تاريخ! بنكتب الرسالة في ملف حقيقي اسمه `message.txt` .
- **ملحوظة للمحترفين:** `writeFileSync` بتوقف الكود لحد ما الملف يتكتب. "ماكس" هيعلمك قدام إن الأفضل نستخدم `writeFile` (النسخة الـ Async) عشان السيرفر يفضل طيارة.

7. الـ Redirect (التحويل)

JavaScript

```
res.statusCode = 302;
res.setHeader("Location", "/");
return res.end();
```

- **لزمتها:** بعد ما خلصنا المصلحة وسيفنا الملف، مش عايزين اليوزر يفضل واقف في صفحة فاضية. بنقول للمتصفح: "ارجع للرئيسية (`/`) وحالة الطلب هي `302` (يعني تحويل مؤقت)".

The Event Loop